Symbiosis Institute of Technology, Pune

Department of Electronics & Telecommunication Engineering
First Year B. Tech Academic Year: 2025-26

EDA Lab Project Report

Analysis on Student Daily Commute

By:

1. Tanmay Agarwal (25070123158)
2. Tannishtha Gupta (25070123049)

Faculty Incharge: Mrs. Snehal Bhosale

ACKNOWLEDGEMENT

We are deeply grateful to our EDA faculty, Mrs. Snehal Bhosale, for her continuous guidance, encouragement, and invaluable feedback throughout the completion of this mini project. Her insights and suggestions greatly enriched the quality and depth of this report.

We also wish to express our sincere gratitude to the college administration at Symbiosis Institute of Technology, Pune, for providing us with the resources, infrastructure, and academic environment necessary for the completion of this project.

We are thankful to our parents and friends for their moral support and motivation throughout the project. Their encouragement was essential in helping us stay focused and committed.

We also acknowledge the open-source Python community and the documentation of libraries such as Pandas, NumPy, Matplotlib, and Seaborn, which served as essential references and learning resources throughout this project.

INTRODUCTION

In this EDA (Exploratory Data Analysis) project, we analyse the daily commute patterns of students across Pune city. Transportation plays a vital role in student life — affecting productivity, daily schedule, and overall well-being. Understanding how students travel, how long they take, how much they spend, and what problems they face is both practically relevant and academically significant.

The dataset, comprising 724 student survey responses collected over one week in March 2026, captures real commute details including mode of transport, travel time, one-way cost, starting point, destination, peak hours, bottlenecks, and satisfaction levels. This data forms the basis of our entire analysis.

The project applies all twelve EDA techniques taught in our Semester 2 curriculum — from basic data loading and descriptive statistics, to cross tabulation, groupby analysis, and data visualisation — using Python and its core data science libraries: Pandas, NumPy, Matplotlib, and Seaborn, executed in a Jupyter Notebook environment.

By integrating a variety of EDA tools and methods, this project offers detailed insights into the daily commute experience of students in Pune, facilitating informed observations and meaningful conclusions about transport efficiency, cost patterns, and student satisfaction.

OBJECTIVE OF THE PROJECT

1. To apply all EDA techniques taught in Semester 2 on a real-world student commute dataset collected via survey.

2. To identify the most and least efficient modes of transport in terms of average travel time and average one-way cost.

3. To uncover the major delay points and bottlenecks faced by students on their daily commute routes across Pune.

4. To understand the distribution of commute patterns by starting point, destination, mode, time of day, and cost.

5. To analyse student satisfaction levels with their daily commute and identify what factors influence satisfaction scores.

6. To present all findings through clear, well-labelled visualisations including bar charts, pie charts, histograms, and horizontal bar charts.

RAW DATA

The dataset for this project was collected through a structured survey shared among students in Pune. Responses were gathered between March 7, 2026 and March 14, 2026. A total of 724 unique responses were recorded across a wide variety of commute scenarios covering seven starting areas and six destination areas in Pune city.

The dataset contains 724 rows and 10 columns. Each row represents one student's one-way commute profile. The columns capture all key aspects of the daily commute experience.

Column Name	Description
Timestamp	Date and time the response was submitted
Starting_Point	Area from which the student commutes (7 unique areas)
Destination	Destination area of the commute (6 unique areas)
Primary_Mode	Main mode of transport used
Secondary_Mode	Secondary mode for multi-leg journeys
Time_of_Day	Time slot of usual commute (4 time slots)
Travel_Time_min	Total one-way travel time in minutes (Numeric)
One_Way_Cost_INR	Total one-way commute cost in Rupees (Numeric)
Delay_Points	Major bottleneck reported on route (132 missing values)
Satisfaction	Overall satisfaction score — 1 (Very Low) to 5 (Very High)

The dataset contains information about 6 primary modes of transport used by students: Metro, Carpool, PMPML City Bus, Cab (Ola/Uber), Auto Rickshaw,

and Personal Two-Wheeler. Starting points covered include Kothrud, Baner, Pashan, Wakad, Shivajinagar, SIT Lavale Campus, and Other areas.

Dataset Property	Value
Total Responses (Rows)	724
Total Features (Columns)	10
Unique Starting Points	7
Unique Destinations	6
Unique Primary Modes	6
Unique Time Slots	4
Missing Values	132 (only in Delay_Points column)
Satisfaction Scale	1 to 5

LIBRARIES USED

Python's data analysis ecosystem provides a powerful set of libraries that enable efficient and effective exploratory data analysis. Each library serves a specific purpose — from data loading and manipulation to numerical computation and visualisation.

The following libraries were used in this project:

1. Pandas
2. NumPy
3. Matplotlib
4. Seaborn

We used the following tools and techniques for Data Analysis:

1. Dataset Loading and Inspection
2. Descriptive Statistics
3. Missing Value Analysis
4. Frequency Count
5. Unique Values
6. Percentage Distribution
7. Filtering Data
8. Cross Tabulation
9. Groupby Analysis
10. Sorting
11. Derived Column
12. Visualisations (Charts)

PANDAS

Pandas is a powerful open-source Python library built on top of NumPy. It provides fast, flexible, and expressive data structures — primarily the DataFrame — designed to work with structured (tabular) data in an intuitive way. It is the primary library used for all data loading, cleaning, manipulation, and analysis operations in this project.

In this project, Pandas was used to load the dataset using pd.read_csv(), rename columns, inspect the data using df.head(), df.tail(), df.info(), df.shape, and df.dtypes, compute descriptive statistics using df.describe(), handle missing values using df.isnull().sum() and df.fillna(), perform frequency counts using value_counts(), analyse unique values using unique() and nunique(), compute percentage distributions, filter data using boolean indexing, perform cross tabulation using pd.crosstab(), group data using groupby(), and sort records using sort_values().

Pandas is the single most important library in this project. Every EDA technique — from inspection to analysis — was performed using Pandas functions.

NUMPY

NumPy (Numerical Python) is the foundational library for numerical computing in Python. It provides support for large multi-dimensional arrays and matrices, along with a wide collection of high-level mathematical functions to operate on these arrays efficiently.

NumPy works as the backbone of Pandas. When Pandas performs operations like mean(), median(), std(), min(), and max() on DataFrame columns, it internally uses NumPy arrays and operations for the computations. NumPy also provides the np.round() function used in our project to round computed values to 2 decimal places for clean, readable output.

In this project, NumPy was imported using 'import numpy as np' and its computational capabilities were leveraged indirectly through Pandas operations for all numerical analyses performed on the Travel_Time_min, One_Way_Cost_INR, and Satisfaction columns.

MATPLOTLIB

Matplotlib is the primary data visualisation library in Python. It provides a comprehensive collection of plotting tools for creating static, animated, and interactive visualisations. Matplotlib gives full control over every element of a chart — from axis labels and titles to colours, line styles, and annotations.

In this project, Matplotlib was used to create all eight visualisations, including bar charts showing mode of transport frequency, histograms showing the distribution of travel time and one-way cost, a donut/pie chart showing the time of day distribution, horizontal bar charts comparing average travel time and cost by mode, and a bar chart displaying the satisfaction rating distribution.

Key Matplotlib functions used in this project include plt.figure() to create figures, plt.hist() for histograms, df['col'].plot(kind='bar') for bar charts, plt.axvline() to draw mean and median reference lines, plt.title(), plt.xlabel(), plt.ylabel() for labels, and plt.show() to display each chart.

SEABORN

Seaborn is a statistical data visualisation library built on top of Matplotlib. It provides a higher-level interface for drawing attractive and informative statistical graphics. Seaborn works seamlessly with Pandas DataFrames and automatically handles many complex formatting tasks, making it easier to create publication-quality visualisations with less code.

Seaborn was imported in this project using 'import seaborn as sns'. While the primary visualisations were built using Matplotlib, Seaborn was imported as it is a standard part of the EDA toolkit and provides additional styling and statistical plot capabilities. Seaborn's color palettes and themes are often used to enhance the visual quality of Matplotlib charts.

Seaborn is particularly useful for creating distribution plots (histplot, kdeplot), categorical plots (boxplot, violinplot), heatmaps, and pair plots — all of which are common tools in exploratory data analysis for understanding relationships and distributions in datasets.

DATASET LOADING AND INSPECTION

The first step in any EDA is to load the dataset and understand its basic structure. We used Pandas to load the CSV file and inspect it using several built-in functions.

pd.read_csv() was used to load the dataset from the CSV file. The column names were then renamed for easier access throughout the notebook.

df.shape returns the dimensions of the DataFrame. Our dataset has the shape (724, 10) — meaning 724 rows (responses) and 10 columns (features).

df.head() displays the first 5 rows of the dataset. This gives a quick preview of the data and helps verify that loading was successful.

df.tail() displays the last 5 rows of the dataset, which confirms that data was loaded completely without truncation.

df.dtypes shows the data type of each column. Timestamp, Starting_Point, Destination, Primary_Mode, Secondary_Mode, Time_of_Day, and Delay_Points are of type String (object). Travel_Time_min, One_Way_Cost_INR, and Satisfaction are of type Integer (int64).

df.info() provides a comprehensive summary of the DataFrame including column names, non-null counts, data types, and memory usage. It confirmed 724 non-null entries in all columns except Delay_Points which had 132 null values.

df.columns returns the list of all column names in the DataFrame, which we used to verify renaming was applied correctly.

DESCRIPTIVE STATISTICS

Descriptive statistics provide a quick numerical summary of the dataset. In this project, df.describe() was applied to generate the statistical summary of all numerical columns — Travel_Time_min, One_Way_Cost_INR, and Satisfaction.

Individual statistical measures were also computed:

Statistic	Travel Time (min)	One-Way Cost (Rs.)
Mean	57.47 min	Rs. 127.00
Median	54.00 min	Rs. 65.50
Std Dev	22.03 min	Rs. 126.11
Minimum	20 min	Rs. 10
Maximum	119 min	Rs. 445
25th %ile	40 min	Rs. 33
75th %ile	72 min	Rs. 210

The mean() function was used to compute the average travel time (57.47 minutes) and average cost (Rs. 127). The median() function returned 54.00 minutes and Rs. 65.50 respectively. The significant difference between mean and median cost indicates a right-skewed distribution — a small number of expensive cab users pull the average above the median.

The std() function computed the standard deviation, round() was used to round values to 2 decimal places, and min() and max() returned the range extremes for both numerical columns.

MISSING VALUE ANALYSIS

Before performing any analysis, it is essential to identify and handle missing (null) values in the dataset. Missing values can distort analysis results and must be addressed appropriately.

df.isnull().sum() was used to detect missing values in each column. The result showed that only the Delay_Points column had 132 missing values. All other 9 columns had zero null entries.

The percentage of missing data was computed as: (df.isnull().sum() / len(df) * 100).round(2). This showed that 18.23% of the Delay_Points column was missing — a significant portion.

To handle this, we used df['Delay_Points'] = df['Delay_Points'].fillna('None') to fill all 132 null values with the string 'None'. This was chosen over dropna() because dropping 132 rows would have reduced the dataset by 18%, causing significant data loss. The value 'None' clearly indicates that the student reported no particular bottleneck — which is itself a meaningful data point.

After filling, df.isnull().sum() was run again to confirm that all columns now had zero missing values, verifying that the dataset was fully clean and ready for analysis.

FREQUENCY COUNT

Frequency count using value_counts() tells us how often each category appears in a column. We applied this to all key categorical columns to understand the distribution of responses.

Primary Mode of Transport	Count	% Share
Metro	131	18.09%
Carpool	130	17.96%
PMPML City Bus	129	17.82%
Cab (Ola/Uber)	121	16.71%
Auto Rickshaw	115	15.88%
Personal Two-Wheeler	98	13.54%

value_counts() was also applied to Time_of_Day (Midday: 205, Night: 177, Morning: 174, Evening: 168), Delay_Points (Waiting for Bus/Metro: 133, None: 132, Hinjewadi Traffic: 127, Yerawada Junction: 114, Chandni Chowk: 110, University Circle: 108), Starting_Point (Kothrud: 129 — highest), Destination (Swargate and Viman Nagar tied at 131 — highest), and Satisfaction (Scores 1 and 5 tied as most frequent at 166 and 165 respectively).

UNIQUE VALUES

The unique() function was used to list all distinct category values present in each categorical column. The nunique() function returned the count of unique values per column.

Unique values found in key columns:

- Starting_Point (7 unique): Other, Kothrud, Pashan, Wakad, SIT Lavale Campus, Shivajinagar, Baner
- Destination (6 unique): Kharadi, Pune Station, Viman Nagar, Swargate, Hinjewadi, Other
- Primary_Mode (6 unique): Metro, Personal Two-Wheeler, Carpool, Cab (Ola/Uber), PMPML City Bus, Auto Rickshaw
- Time_of_Day (4 unique): Morning Peak, Midday, Evening Peak, Night
- Delay_Points (6 unique): Chandni Chowk, Hinjewadi Traffic, None, University Circle, Yerawada/Kalyani Nagar Junction, Waiting for Bus/Metro
- Satisfaction (5 unique): 1, 2, 3, 4, 5

df.nunique() was applied to the entire DataFrame to get a comprehensive view of distinct values per column. Travel_Time_min had 92 unique values and One_Way_Cost_INR had 249 unique values — reflecting the continuous nature of these numerical columns.

PERCENTAGE DISTRIBUTION

Percentage distribution was computed using value_counts(normalize=True) * 100. This expresses each category's frequency as a percentage of the total, enabling fair comparison across groups regardless of total count.

Percentage distribution of Time_of_Day: Midday (11AM-4PM) 28.31%, Night (After 8PM) 24.45%, Morning Peak (8-11AM) 24.03%, Evening Peak (4-8PM) 23.20%. The near-even distribution across all four time slots suggests that student commuting is spread throughout the entire day.

Percentage distribution of Satisfaction: Score 1 (Very Low) 22.93%, Score 2 (Low) 19.20%, Score 3 (Neutral) 17.40%, Score 4 (High) 17.68%, Score 5 (Very High) 22.79%. The near-equal split between extreme scores (1 and 5) reveals a bimodal distribution — indicating that student satisfaction is highly polarised.

Percentage distribution of Delay_Points: Waiting for Bus/Metro 18.37%, None (no delay) 18.23%, Hinjewadi Traffic 17.54%, Yerawada/Kalyani Nagar Junction 15.75%, Chandni Chowk 15.19%, University Circle 14.92%. Almost evenly spread bottlenecks indicate that delays are a systemic, multi-point problem across the city.

FILTERING DATA

Filtering using boolean indexing allows us to isolate specific subsets of the data based on a condition. This enables targeted analysis of particular groups or scenarios.

The following filters were applied in this project:

1. Metro Users: df[df['Primary_Mode'] == 'Metro'] — Returned 131 students who use Metro as their primary mode. This is the largest single-mode group.

2. Morning Peak Commuters: df[df['Time_of_Day'] == 'Morning Peak (8 AM - 11 AM)'] — Returned 174 commutes during the traditional rush hour period.

3. High-Cost Trips (> Rs.300): df[df['One_Way_Cost_INR'] > 300] — Identified expensive commutes, predominantly Cab and Auto Rickshaw users.

4. Long Commutes (> 90 min): df[df['Travel_Time_min'] > 90] — Identified the most time-burdened students, mostly PMPML City Bus users.

5. Students from Kothrud: df[df['Starting_Point'] == 'Kothrud'] — Returned 129 students, the highest count from any single starting area.

CROSS TABULATION

Cross tabulation using pd.crosstab() produces a frequency table showing the relationship between two categorical variables simultaneously. It reveals patterns that individual frequency counts cannot capture.

The following five cross tabulations were performed:

1. Primary Mode vs Time of Day: This crosstab revealed how transport choices vary across time slots. Metro peaks in the Morning (37 users) and Evening (35 users) — aligning with peak college and office hours. PMPML Bus sees highest usage at Night (38) — likely due to students returning late after college hours.

2. Starting Point vs Primary Mode: This revealed area-specific transport preferences. Wakad has the highest Auto Rickshaw usage (21) reflecting limited metro access. Kothrud has the highest PMPML Bus usage (27) due to dense bus route coverage.

3. Destination vs Primary Mode: Kharadi sees the highest Metro usage (35) reflecting strong metro connectivity to that IT hub. Swargate and Viman Nagar show higher Carpool usage.

4. Primary Mode vs Satisfaction: PMPML City Bus and Carpool users gave the most Score-5 (Very High) ratings (31 each). Auto Rickshaw had the highest Score-1 (Very Low) count (34) — reflecting dissatisfaction with erratic pricing and availability.

5. Time of Day vs Delay Points: Midday commutes see the highest 'Waiting for Bus/Metro' delays (46) — suggesting reduced transport frequency outside peak hours is a major pain point.

GROUPBY ANALYSIS

The groupby() function in Pandas groups rows by a specified column and then applies aggregate functions like mean(), count(), max(), or sum() to compute summary statistics for each group.

The following groupby operations were performed in this project:

Primary Mode	Avg Time (min)	Avg Cost (Rs.)	Avg Satisfaction	Max Cost
Metro	38.68	23	3.0	445
Carpool	53.52	69	3.1	445
Personal Two-Wheeler	53.28	68	2.9	445
Auto Rickshaw	58.09	286	2.8	445
Cab (Ola/Uber)	59.27	298	3.0	445
PMPML City Bus	81.47	33	3.1	445

The groupby analysis clearly shows Metro as the winner on both dimensions — it is simultaneously the fastest (38.68 min avg) and cheapest (Rs. 23 avg) mode of transport. PMPML City Bus is the slowest (81.47 min avg) but very affordable (Rs. 33 avg). Cab is the most expensive at Rs. 298 on average.

SORTING

The sort_values() function in Pandas sorts a DataFrame by one or more columns in ascending or descending order. It helps identify extreme values and rank records by a particular metric.

In this project, the following sorting operations were performed:

1. Sorted by Travel Time (Ascending): df.sort_values('Travel_Time_min').head(10) — This revealed the 10 shortest commutes. The shortest trip was just 20 minutes, by a Metro user travelling from Shivajinagar to Swargate.

2. Sorted by One-Way Cost (Descending): df.sort_values('One_Way_Cost_INR', ascending=False).head(10) — This revealed the 10 most expensive commutes. The maximum one-way cost was Rs. 445, paid by Cab users on longer outer-area routes.

3. Sorted by Satisfaction (Descending): df.sort_values('Satisfaction', ascending=False).head(10) — This revealed commutes with the highest satisfaction scores. Score 5 (Very High) was found across all modes, suggesting that satisfaction is not mode-dependent alone but is also influenced by personal expectations and route familiarity.

DERIVED COLUMN

A derived column is a new column created from existing data using transformations or custom logic. In this project, we created a new column called Commute_Category using the apply() function with a custom classification function.

The classification function categorised each commute into one of three categories based on Travel_Time_min:

- Short (<=30 min): Commutes of 30 minutes or less
- Medium (31-60 min): Commutes between 31 and 60 minutes
- Long (>60 min): Commutes longer than 60 minutes

The distribution of the Commute_Category column was:

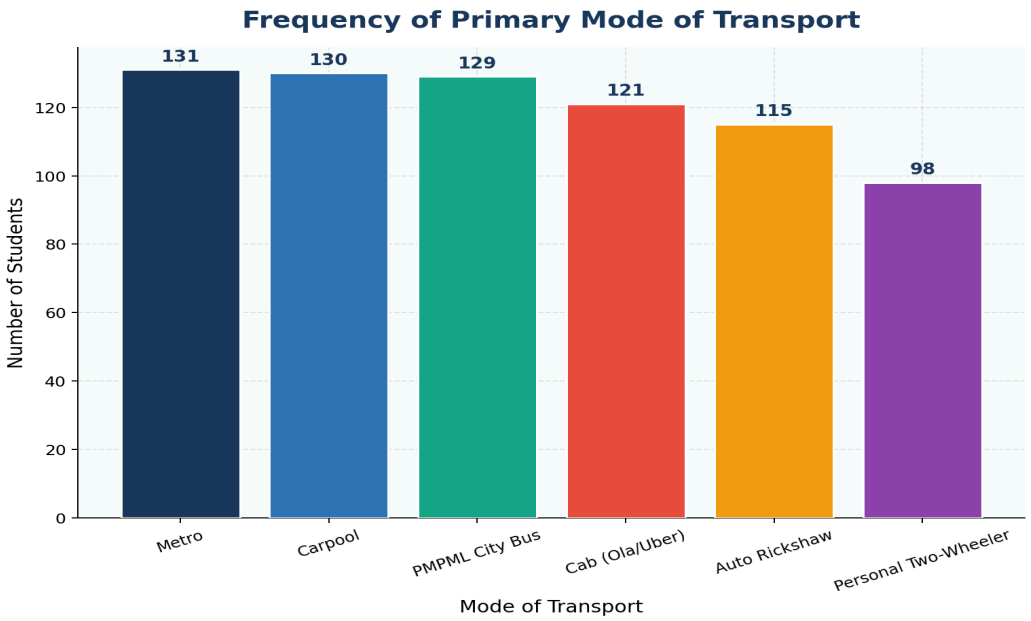
Category	Count	Percentage
Short (<=30 min)	109	15.06%
Medium (31-60 min)	317	43.78%
Long (>60 min)	298	41.16%

An overwhelming 84.94% of students commute for more than 30 minutes one-way. This finding highlights that long commutes are the norm rather than the exception for students in Pune.

VISUALISATIONS (CHARTS)

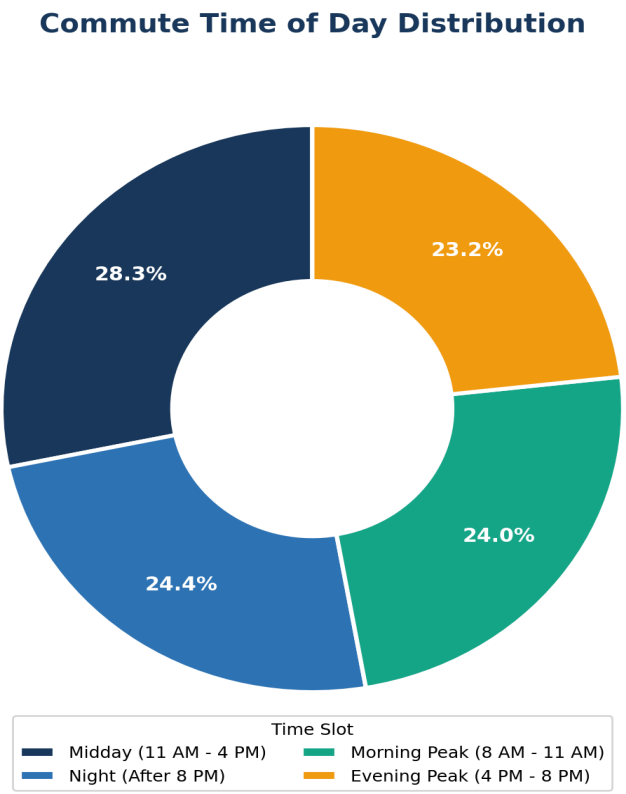
Visual representations in the form of charts and graphs are used to illustrate trends and patterns in the data clearly. The following eight charts were generated using Matplotlib and Seaborn.

Chart 1: Frequency of Primary Mode of Transport



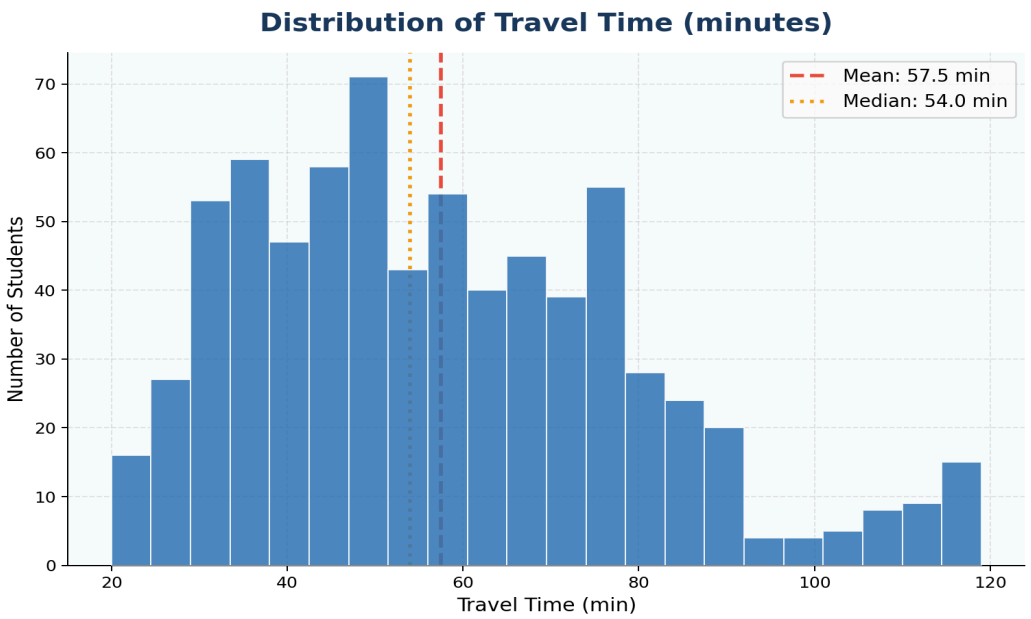
This bar chart shows the count of students using each primary mode of transport. Metro is the most used mode (131 students, 18.09%) while Personal Two-Wheeler is the least used (98 students, 13.54%). All six modes are surprisingly close in usage — within a 5% range of each other — indicating that students across Pune rely on diverse transport options.

Chart 2: Commute Time of Day Distribution



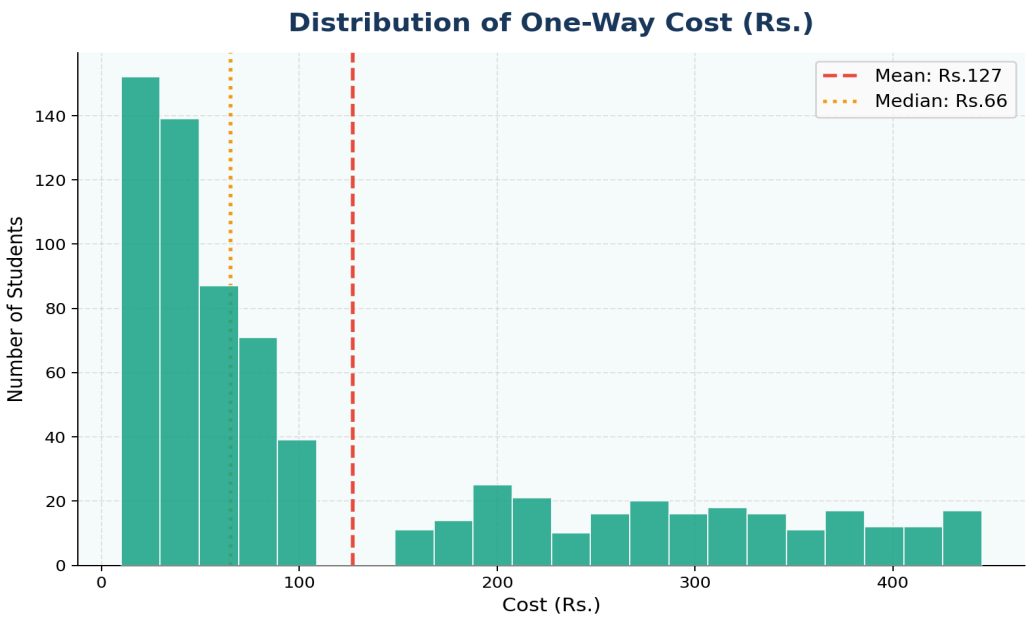
This donut chart shows the percentage of commutes in each time slot. Midday (11AM-4PM) is the most common with 28.31%, followed by Night (24.45%), Morning Peak (24.03%), and Evening (23.20%). The near-equal distribution across all slots indicates that student commuting is spread throughout the day rather than concentrated in traditional peak hours.

Chart 3: Distribution of Travel Time (minutes)



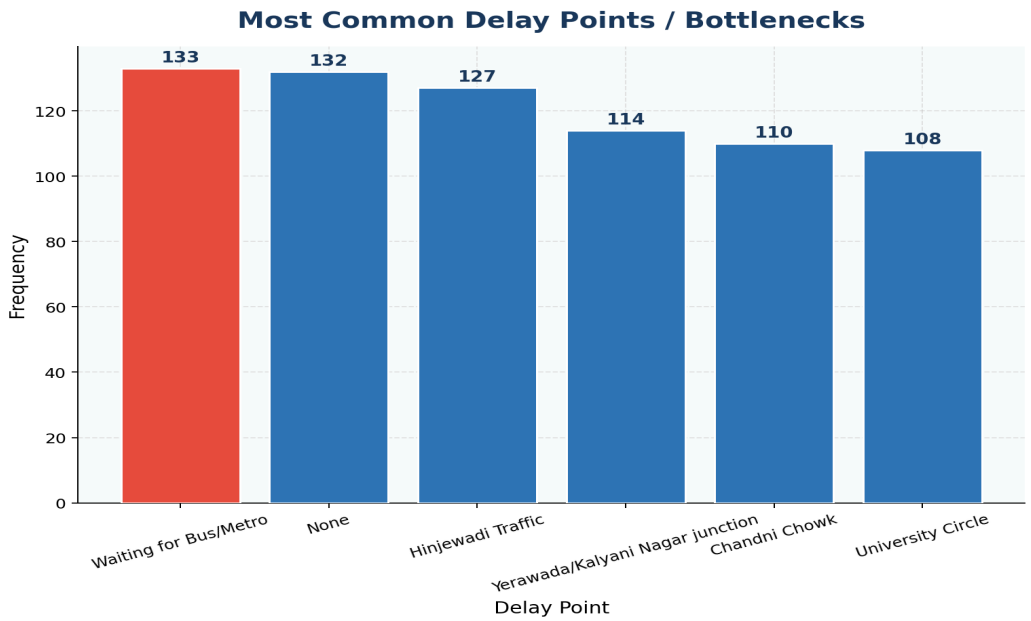
This histogram shows the distribution of one-way travel time across all 724 commutes. The distribution is approximately normal with a slight right skew. The mean (57.47 min, shown by the dashed red line) is close to the median (54 min, shown by the dotted orange line), indicating a relatively symmetric distribution. Most students travel between 35 to 75 minutes one-way.

Chart 4: Distribution of One-Way Commute Cost (Rs.)



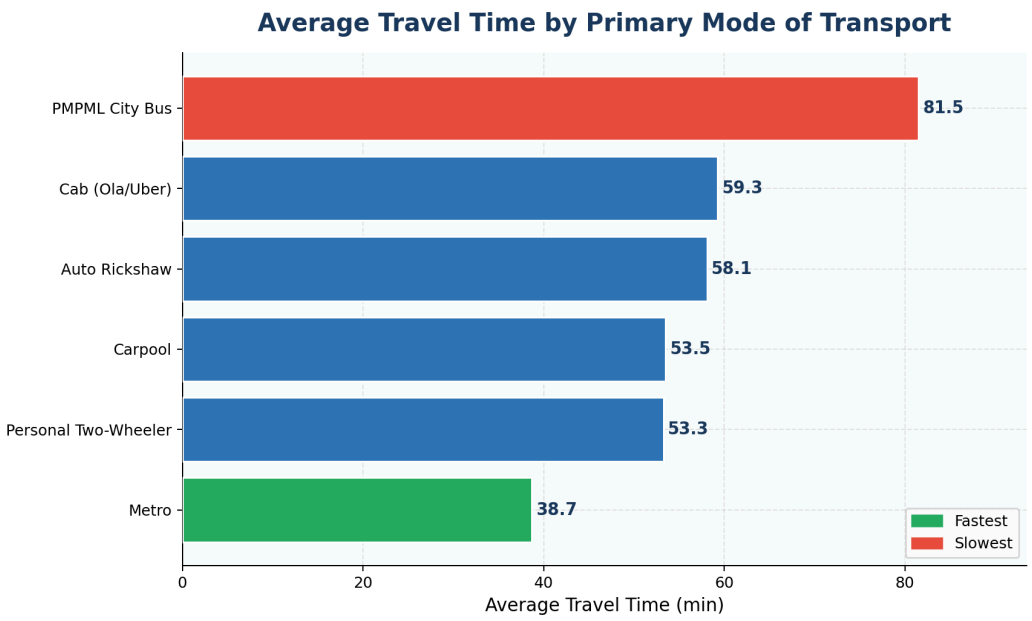
This histogram shows the distribution of one-way commute cost. Unlike travel time, the cost distribution is heavily right-skewed. The median (Rs. 65.50) is much lower than the mean (Rs. 127), confirming a long right tail caused by a relatively small number of Cab users paying Rs. 300-445 per trip. The majority of students spend under Rs. 100 one-way.

Chart 5: Most Common Delay Points / Bottlenecks



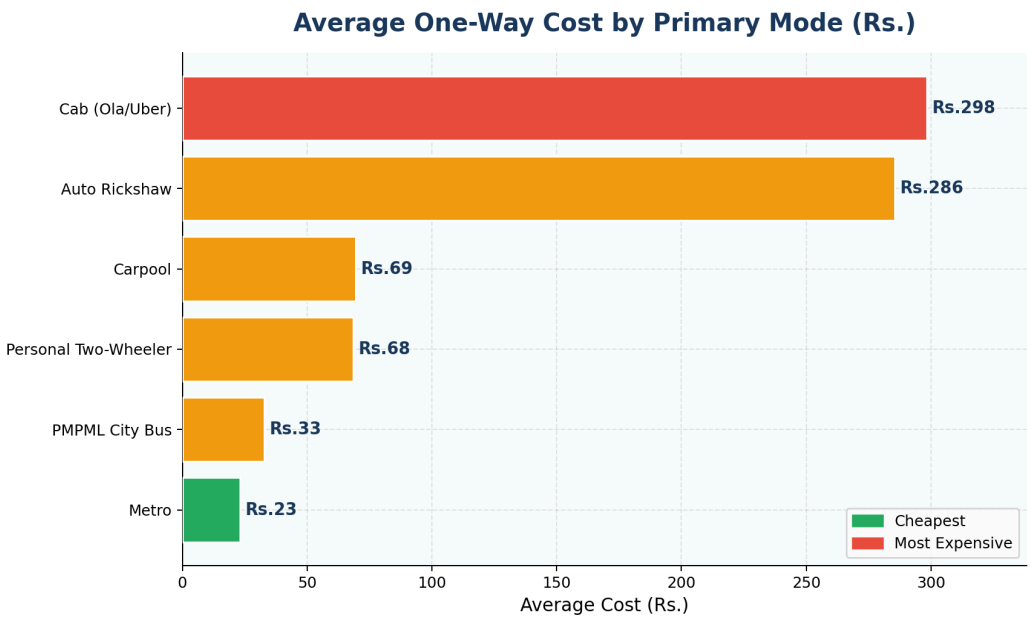
This bar chart shows the frequency of each delay point reported by students. 'Waiting for Bus/Metro' is the single most common bottleneck with 133 cases, highlighted in red. This finding directly points to a service frequency issue with public transport. 132 students reported no delay ('None'). The remaining bottlenecks — Hinjewadi Traffic (127), Yerawada Junction (114), Chandni Chowk (110), and University Circle (108) — are all major road junctions in Pune known for traffic congestion.

Chart 6: Average Travel Time by Primary Mode of Transport



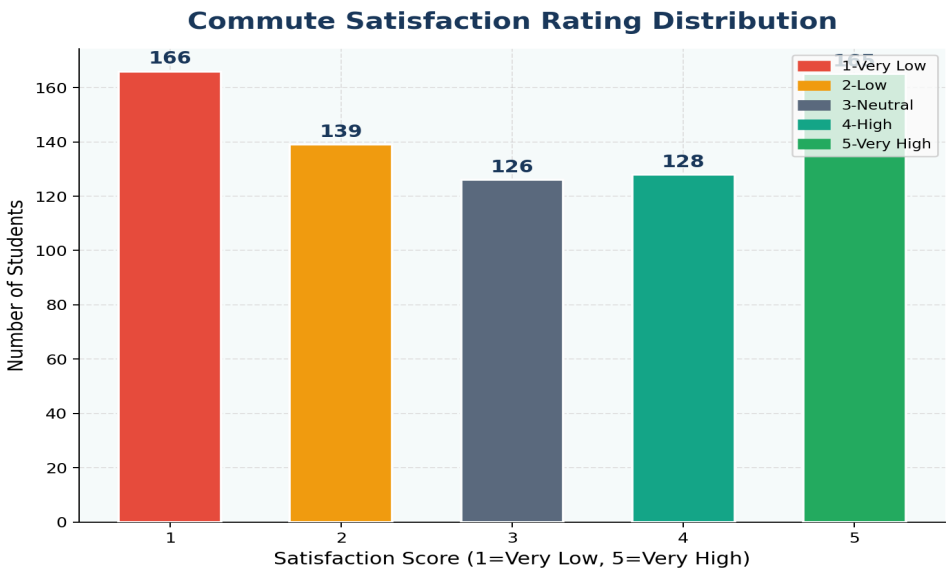
This horizontal bar chart compares the average one-way travel time for each mode. Metro is the fastest at 38.68 minutes (shown in green), while PMPML City Bus is the slowest at 81.47 minutes (shown in red) — more than double Metro's average time. This is likely because buses share road space with other vehicles and make many intermediate stops.

Chart 7: Average One-Way Cost by Primary Mode (Rs.)



This horizontal bar chart compares the average one-way cost for each mode. Metro is the cheapest at Rs. 23 average (shown in green), while Cab (Ola/Uber) is the most expensive at Rs. 298 average (shown in red) — 13 times more expensive than Metro. Taken together with Chart 6, Metro is clearly the dual winner — both fastest and cheapest simultaneously.

Chart 8: Commute Satisfaction Rating Distribution



This bar chart shows how satisfaction scores are distributed across all 724 responses. The distribution is bimodal — Score 1 (Very Low) and Score 5 (Very High) are nearly tied as the most common ratings, at 166 and 165 students respectively. The mean satisfaction score is 2.98 — nearly neutral. This strongly polarised pattern suggests that commute experience is highly subjective and mode-dependent.

CONCLUSION

Through this EDA mini project on student daily commute data, the following key conclusions can be drawn:

Finding	Result
Most Used Mode	Metro (18.09% — 131 students)
Fastest Mode (avg)	Metro — 38.68 minutes
Slowest Mode (avg)	PMPML City Bus — 81.47 minutes
Cheapest Mode (avg)	Metro — Rs. 23 per trip
Most Expensive Mode (avg)	Cab (Ola/Uber) — Rs. 298 per trip
Average Travel Time	57.47 minutes one-way
Average One-Way Cost	Rs. 127
Biggest Bottleneck	Waiting for Bus/Metro — 133 cases
Peak Origin Area	Kothrud — 129 students
Long Commutes (>60 min)	298 students — 41.16% of all responses
Missing Values Handled	132 nulls in Delay_Points — filled with 'None'
Satisfaction Pattern	Bimodal — Score 1 and Score 5 most frequent

Metro stands out as the best mode of transport for students in Pune — it is simultaneously the fastest, the most affordable, and the most popular. PMPML City Bus, while very economical at Rs. 33 average, imposes a heavy time burden at 81 minutes average. Cab services offer convenience but at a steep cost of Rs. 298 — making them unsuitable for daily student use.

An overwhelming 85% of students travel more than 30 minutes one-way — highlighting that long commutes are the norm for Pune students, not an exception. The top bottleneck — Waiting for Bus/Metro — points clearly to a public transport frequency problem that, if addressed, could significantly improve student commute satisfaction.

This project successfully demonstrated the application of all twelve EDA techniques taught in Semester 2 on a real-world dataset. The complete EDA pipeline — from data loading, inspection, and cleaning to analysis, grouping, filtering, and visualisation — was implemented in Python using Pandas, NumPy, Matplotlib, and Seaborn in a Jupyter Notebook environment.